

ML on Big Data

Naive Bayes multinomial en MapReduce sur Spark

Abdou1818

Master — Université Paris-Dauphine

Juillet 2026

Dépôt : <https://github.com/Abdou1818/naive-bayes-mapreduce-spark>

1 Objectif

Implémenter le classifieur **Naive Bayes multinomial** selon le paradigme **MapReduce** sur Apache Spark, en **deux versions** :

- une version **RDD** (API bas niveau, `map` / `reduceByKey`) ;
- une version **DataFrames** (API relationnelle, `explode` + `groupBy` / `agg`).

Les deux versions doivent produire des **prédictions rigoureusement identiques**, et l'on mène une **étude de scalabilité** faisant varier (i) le volume de données et (ii) le parallélisme (`local[k]`).

2 Données

Jeu	Rôle	Taille	Source
SMS Spam Collection	démonstration / test (petit)	5 574 SMS (ham/spam)	UCI
20 Newsgroups	scalabilité (grand)	18 846 docs, 20 classes	scikit-learn

Pour 20 Newsgroups, les en-têtes, pieds de page et citations sont retirés (`remove=("headers", "footers", "quotes")`) afin de ne garder qu'un signal textuel et d'éviter un sur-apprentissage trivial.

3 Rappel du modèle

On travaille en **sac de mots** (bag-of-words). Pour un document d , une classe c , un vocabulaire de taille V et un lissage de Laplace $\alpha = 1$, le modèle multinomial pose :

$$\log P(c) = \log \left(\frac{N_c}{N} \right), \quad (1)$$

$$\log P(w \mid c) = \log \left(\frac{\text{count}_{w,c} + \alpha}{\text{total}_c + \alpha V} \right), \quad (2)$$

où N est le nombre de documents d'entraînement, N_c les documents de la classe c , $\text{count}_{w,c}$ les occurrences du mot w dans la classe c , et $\text{total}_c = \sum_w \text{count}_{w,c}$.

La **prédiction** maximise, en logarithme (pour éviter les sous-débordements) :

$$\text{score}(d, c) = \log P(c) + \sum_{w \in d} \text{count}_w(d) \log P(w \mid c). \quad (3)$$

Notre implémentation reproduit exactement `sklearn.naive_bayes.MultinomialNB` ($\alpha = 1$, `fit_prior=True`), ce qui est **vérifié empiriquement** (§6).

4 Formulation MapReduce

L'entraînement se réduit à des **comptages** — le cas idéal pour MapReduce. Le cœur est le calcul de $\text{count}_{w,c}$ (occurrences de chaque mot par classe).

4.1 Version RDD (`src/nb_rdd.py`)

Chaque document est une paire (`indice_classe`, [`indices de mots`]).

- **MAP** (`flatMap`) : pour chaque occurrence d'un mot w dans un document de classe c , émettre $((c, w), 1)$.
- **REDUCE** (`reduceByKey(add)`) : additionner les $1 \rightarrow \text{count}_{w,c}$.

On compte de même les documents par classe (N_c) et les tokens par classe (total_c).

4.2 Version DataFrames (`src/nb_dataframe.py`)

Le DataFrame a le schéma (`label:int`, `words:array<int>`).

- **MAP** : `explode(words) → une ligne (label, word) par occurrence` (équivalent du `flatMap`).
- **REDUCE** : `groupBy(label, word).count() → countw,c`.

4.3 Construction du modèle et broadcast

Les deux versions produisent **exactement les mêmes agrégats entiers**, puis appellent la **même** fonction `nb_common.build_model` (comptes $\rightarrow$ log-probabilités lissées) et la **même** fonction `nb_common.predict_indices`. Comme les comptes sont entiers (reproductibles au bit près) et que la prédiction est déterministe, **les deux versions donnent des prédictions identiques par construction**.

Le modèle est stocké de façon **creuse mais équivalente au dense** : on ne mémorise $\log P(w | c)$ que pour les couples (c, w) de compte non nul, plus une valeur par défaut $\log(\alpha/(\text{total}_c + \alpha V))$ pour les mots de compte nul. Ce modèle est **broadcasté** (`sc.broadcast`) : Spark l'envoie une seule fois par worker (lecture seule), au lieu de le re-sérialiser à chaque tâche — essentiel quand $V \times C$ est grand. La prédiction se fait via un `map` (RDD) ou une **UDF** (DataFrame) référençant le modèle broadcasté.

5 Protocole expérimental

- **Machine** : macOS (Apple Silicon, arm64), **8 cœurs** ($4 \text{ performance} + 4 \text{ efficiency}$).
- **Pile** : Python 3.12, PySpark 3.5.8, JDK 17. Spark en mode **local** (`local[k]`).
- **Deux expériences** (`src/benchmark.py`) :
 1. **Volume** : à `local[*]` (8 threads), réplication de 20 Newsgroups par un facteur $\in \{ \times 1, \times 2, \times 4, \times 8 \}$.
 2. **Parallélisme** : à volume fixe (facteur $\times 8$, ~ 121 k documents), variation de `local[k]`, $k \in \{1, 2, 4, 8\}$.
- **Précautions méthodologiques** (corrigées après une première version) :
 - **Anti-fuite de données** : la réplication est faite **après** le découpage train/test, et **séparément** de chaque côté. Un document de test ne peut donc jamais apparaître (même dupliqué) dans l'entraînement.

- **Bruit de mesure** : option `--repeat 3` — chaque point est mesuré 3 fois et agrégé en **moyenne $\pm$ écart-type** (colonnes `train_time_std`, `predict_time_std`; barres d'erreur sur les graphes).

```
python src/benchmark.py --factors 1 2 4 8 --cores 1 2 4 8 --repeat 3
```

6 Validation de correction

- **Smoke test** (`tests/test_smoke.py`, aussi en CI GitHub Actions) : sur un mini-jeu, RDD et DataFrame donnent **accuracy et prédictions identiques**.
- **Notebook** (`notebooks/demo.ipynb`) sur SMS Spam réel : **accuracy** = 0.9848, avec **RDD = DataFrame = `sklearn.MultinomialNB`** (prédictions identiques).
- Le benchmark **assert** à chaque point l'égalité des accuracy RDD/DataFrame.

7 Résultats

20 Newsgroups complet, `--repeat 3`; temps en secondes, moyenne $\pm$ écart-type.

7.1 Scalabilité en volume (`local[*]` = 8 threads)

Facteur	n_train	RDD train	DF train	RDD pred	DF pred	Acc.
$\times 1$	15 076	1.81 ± 0.67	3.07 ± 1.25	0.44 ± 0.04	1.39 ± 0.27	0.620
$\times 2$	30 152	1.89 ± 0.15	3.10 ± 0.20	0.68 ± 0.04	2.01 ± 0.19	0.677
$\times 4$	60 304	3.16 ± 0.15	5.11 ± 0.25	1.26 ± 0.03	3.42 ± 0.21	0.704
$\times 8$	120 608	5.27 ± 0.11	8.57 ± 0.09	2.29 ± 0.09	6.91 ± 1.10	0.711

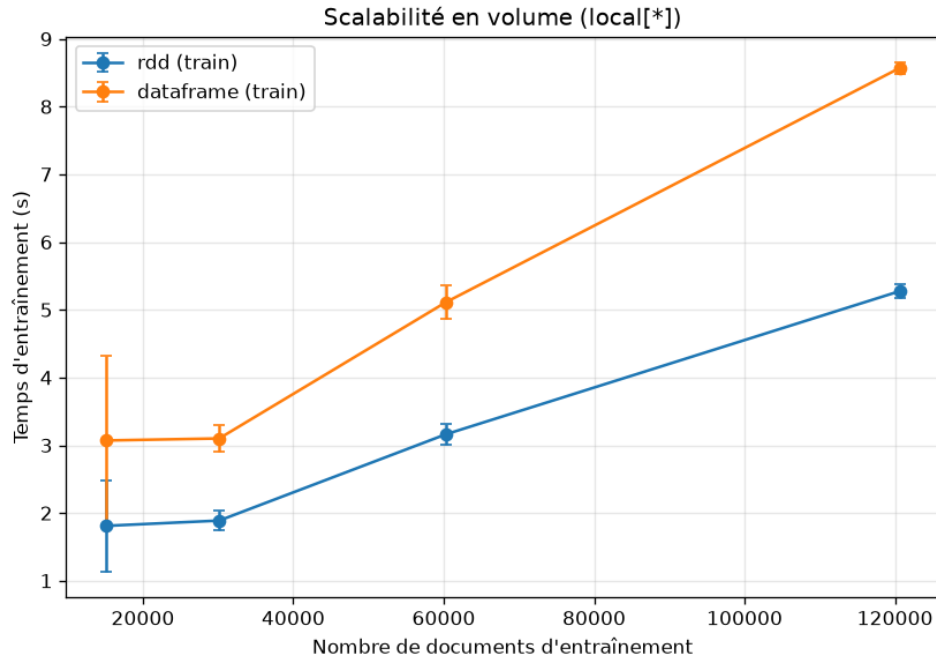


FIGURE 1 – Temps d'entraînement en fonction du volume de données (`local[*]`).

- De $\times 1$ à $\times 8$ (**$8\times$ de données**), le temps d'entraînement n'augmente que d'un facteur ~ 2.9 : **passage à l'échelle sous-linéaire**, très favorable.
- **RDD est plus rapide que DataFrame à tous les points**, avec un écart nettement supérieur aux écarts-types.
- La **prédiction DataFrame** est $\sim 3\times$ plus lente que la RDD : l'UDF Python est invoquée et sérialisée **ligne par ligne**.

7.2 Scalabilité en parallélisme (facteur $\times 8$, ~ 121 k docs)

Speed-up = $T(1)/T(k)$.

Cœurs	RDD train	speed-up	RDD pred	DF train	DF pred
1	12.66 ± 0.30	$1.00\times$	5.00 ± 0.04	10.05 ± 0.07	12.07 ± 0.36
2	7.23 ± 0.15	$1.75\times$	3.34 ± 0.08	9.13 ± 0.30	8.63 ± 0.02
4	5.88 ± 0.57	$2.15\times$	2.68 ± 0.28	9.18 ± 0.27	8.09 ± 0.53
8	10.97 ± 1.51	$1.15\times$	3.48 ± 0.51	12.55 ± 1.45	8.30 ± 0.85

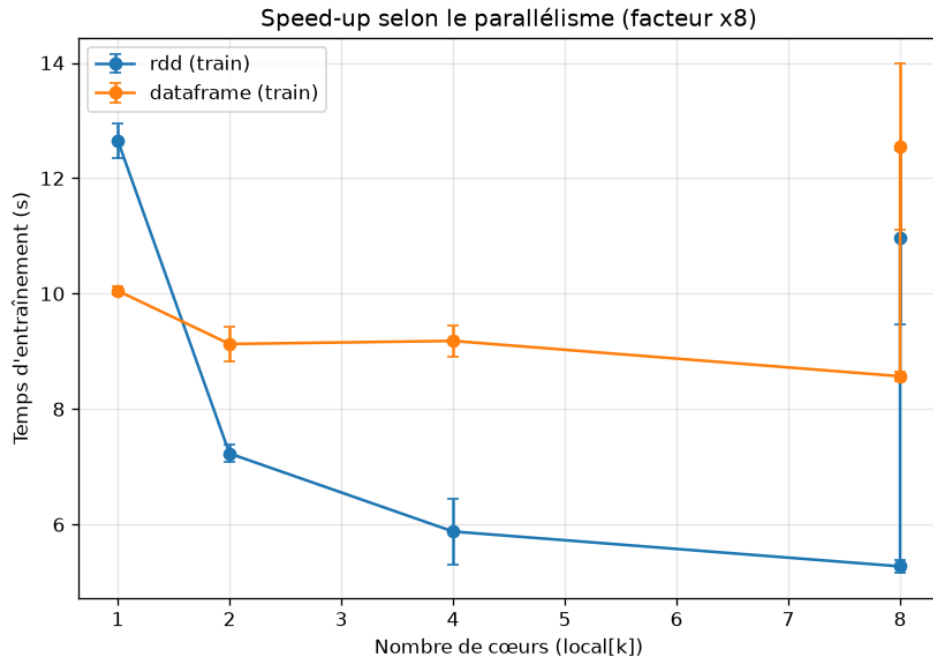


FIGURE 2 – Temps d'entraînement en fonction du nombre de cœurs (facteur $\times 8$).

- **RDD** : le speed-up progresse jusqu'à $\sim 2.15\times$ à 4 cœurs, puis **régresse à 8 cœurs**. Explication matérielle : sur Apple Silicon, `local[8]` embarque les 4 cœurs *efficiency* (plus lents) et met le driver en contention $\rightarrow$ point plus lent et fortement bruité (± 1.5 s).
- **DataFrame** : ne bénéficie quasiment pas des cœurs supplémentaires (Catalyst parallélise déjà en interne ; l'overhead SQL + UDF domine).

8 Discussion

8.1 RDD vs DataFrames

Sur ce problème (comptages + prédiction par UDF), **RDD est plus rapide** que DataFrames, surtout en prédiction. L'API DataFrames reste plus lisible et bénéficierait davantage à plus

grande échelle ou avec des fonctions natives Spark (sans UDF Python). Le point clé pédagogique — **prédictions identiques** — est vérifié.

8.2 Speed-up sous-linéaire

Le mode `local` fait tourner driver et exécuteurs dans **une seule JVM** ; « cœurs » = threads. Les overheads (shuffle, sérialisation, warmup, GC) et l'hétérogénéité des cœurs Apple Silicon expliquent un speed-up **bien en deçà du linéaire idéal** ($\sim 2\times$ utiles sur 4 cœurs performance).

8.3 Accuracy et réplication — un effet de lissage

Après correction de la fuite, l'accuracy **ne monte plus fortement** avec le volume, mais elle **n'est pas parfaitement constante** ($0.620 \rightarrow 0.711$). Ce n'est **pas** de la fuite : répliquer le *train* multiplie tous les comptes par le facteur f , si bien que $\log P(w | c) = \log ((f \cdot \text{count} + \alpha) / (f \cdot \text{total} + \alpha V))$ — le lissage $\alpha = 1$ devient **relativement négligeable** quand les comptes gonflent, donc le modèle est moins régularisé. La valeur de référence « honnête » est celle à $\times 1 = 0.620$. Pour une accuracy strictement plate, il faudrait mettre à l'échelle le lissage ($\alpha = f$) — mais l'étude visant les **temps**, on conserve $\alpha = 1$ et l'on documente l'effet.

9 Limites

- **Mode local uniquement** : pas de vrai cluster ; conclusions valables pour un nœud multi-cœurs.
- **Cœurs hétérogènes** (Apple Silicon) : le point `local` [8] est peu fiable.
- **Prédiction DataFrame par UDF Python** : goulot d'étranglement.
- **Réplication** comme proxy de « big data » : gonfle le volume mais pas la diversité ; interagit avec le lissage.

10 Conclusion et pistes

Le classifieur Naive Bayes multinomial a été implémenté en MapReduce sur Spark en deux versions **cohérentes** (prédictions identiques, alignées sur scikit-learn), avec une étude de scalabilité rigoureuse (anti-fuite, mesures répétées). Les résultats montrent un **scaling en volume sous-linéaire**, un **avantage de la version RDD**, et un **speed-up parallèle réel mais modéré** limité par le mode local et le matériel.

Pistes : exécution sur cluster (YARN/K8s), prédiction DataFrame sans UDF Python, métriques par classe (précision/rappel, matrice de confusion), comparaison temps Spark vs scikit-learn sur petit volume, mise à l'échelle du lissage.

11 Reproductibilité

```
python3.12 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

python tests/test_smoke.py           # egalite RDD == DataFrame
python data/download_sms_spam.py     # petit jeu (notebook)
python data/download_20newsgroups.py # grand jeu (benchmark)
jupyter lab notebooks/demo.ipynb    # demo + comparaison sklearn

python src/benchmark.py --factors 1 2 4 8 --cores 1 2 4 8 --repeat 3
# -> results/results.csv + results/scalability_{datasize,cores}.png
```

Prérequis : Python 3.10–3.12 (pas 3.13+ pour pyspark 3.5) et un JDK 8/11/17 (pas 20+). Sur macOS, `get_spark()` sélectionne automatiquement le JDK 17.